

PROJECT REQUIREMENT TO BUILD

You are the lead full-stack AI engineer responsible for building the entire project from start to finish.

PROJECT

Build a complete, working, production-style student project called:

KnowledgeGuard AI

Subtitle:

An Agentic RAG System for Detecting Outdated and Conflicting Knowledge in AI Systems

This is NOT a simple chatbot, PDF chatbot, document summarizer, or basic LLM API project.

The application must demonstrate a genuine combination of:

- **1 LLM**
- **1 AI Agent**
- **RAG (Retrieval-Augmented Generation)**

All three are compulsory.

Your responsibility is to design, implement, run, test, debug, and polish the entire project.

Do not just generate code or a project skeleton.

Actually build the working application.

1. CORE PROBLEM

Modern AI systems increasingly depend on external knowledge through documents, databases, RAG systems, and organizational knowledge bases.

However, knowledge changes over time.

For example:

Old knowledge:

"API v2 is recommended."

Newer knowledge:

"API v2 has been deprecated. New applications should use API v3."

A normal RAG system may retrieve both documents, but it does not necessarily perform a dedicated investigation of whether the existing knowledge has become outdated or contradictory.

KnowledgeGuard AI solves this problem.

It acts as a:

Knowledge Reliability Layer for AI Systems

The system investigates stored knowledge against available evidence and identifies whether knowledge appears to be:

- CURRENT
- OUTDATED
- CONFLICTING
- UNCERTAIN

It must provide:

- evidence
- sources
- dates
- versions
- reasoning
- confidence
- recommendation
- human verification requirement

IMPORTANT:

Do NOT claim that the AI determines absolute truth.

The system should say that it detects **potential knowledge decay, inconsistency, or insufficient evidence based on retrieved sources.**

2. MAIN CONCEPT

The entire application should follow this principle:

RAG

Finds the evidence.

AI Agent

Investigates the evidence.

LLM

Reasons about the evidence.

KnowledgeGuard AI

Reports the knowledge status and recommends human verification when needed.

3. DO NOT ASK ME TO BUILD PARTS MANUALLY

You should independently:

- create folders
- create files
- install dependencies
- configure the project
- implement frontend
- implement backend
- implement RAG
- implement embeddings
- implement vector database
- implement AI Agent
- connect the LLM
- implement database
- create sample documents
- implement dashboard
- implement investigation workflow
- test the application
- fix errors

- improve UI
- create README
- prepare the application for demonstration

If a reasonable technical decision is required, make the decision yourself.

Do not repeatedly ask me which library to use.

Prefer simple, reliable technologies that can be completed within 3 days.

4. TECHNOLOGY STACK

Use the following stack unless there is a strong technical reason to change it.

Frontend

React + Vite

Use:

- React
- Tailwind CSS
- modern responsive UI
- Lucide icons or another lightweight icon library

Backend

Python + FastAPI

LLM

Use ONE LLM provider/model.

The model must be configurable through .env.

Do not hard-code API keys.

AI Agent

Use:

LangGraph

Use one agent:

Knowledge Investigator Agent

RAG

Use:

- document chunking
- embeddings
- vector retrieval
- metadata filtering
- retrieved evidence

Use ChromaDB or FAISS.

Choose whichever is simpler and more reliable in this environment.

Database

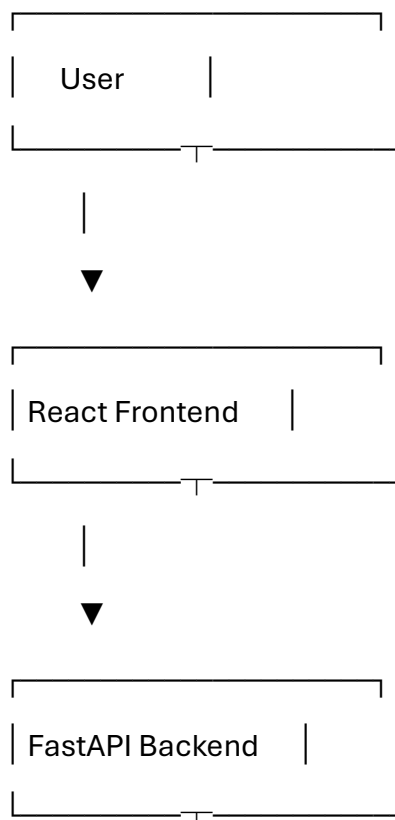
Use SQLite for the MVP.

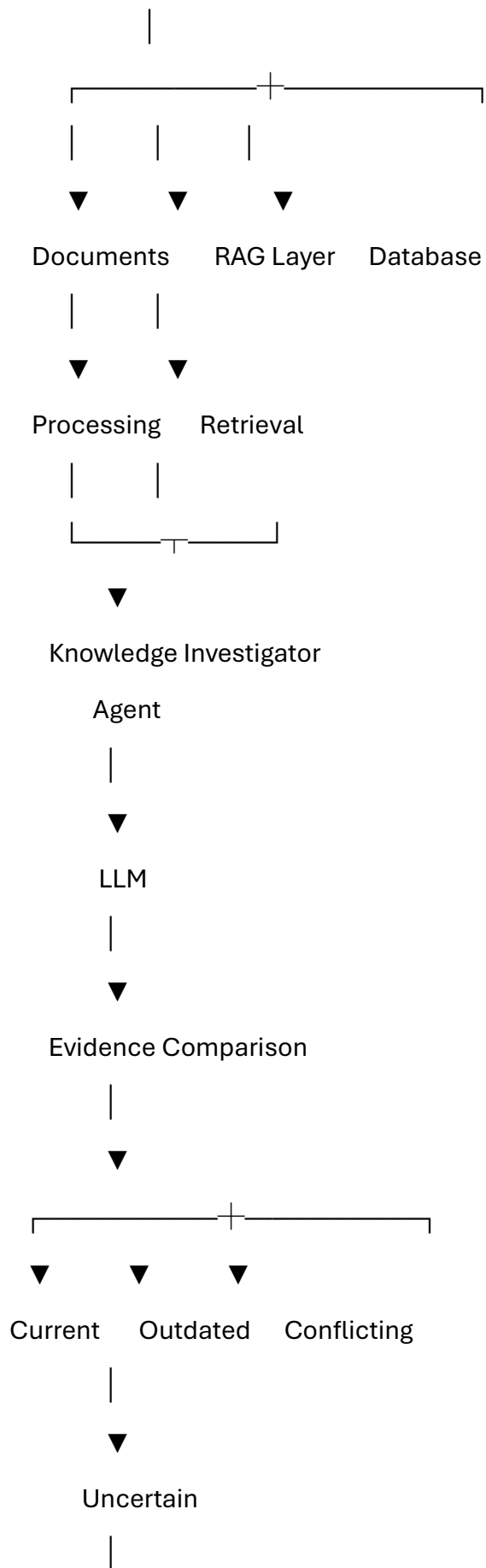
Use SQLAlchemy or another clean ORM if useful.

Do not introduce PostgreSQL unless genuinely required.

5. APPLICATION ARCHITECTURE

Implement this architecture:







Human Verification

6. REQUIRED FEATURES

A. Dashboard

Create a professional dashboard called:

KnowledgeGuard AI

Display real statistics from the database.

Cards:

- Total Knowledge Items
- Current
- Outdated
- Conflicting
- Uncertain
- Requires Review

Do NOT use permanently hard-coded statistics.

The numbers must update based on actual investigations/documents.

Also show:

- recent investigations
- recently uploaded documents
- knowledge requiring review
- knowledge status distribution

7. KNOWLEDGE BASE

Create a Knowledge Base page.

Users should be able to upload:

- PDF
- TXT

DOCX can be added if it does not complicate the build.

For every uploaded document store:

- filename
- source
- version
- document date
- topic
- upload date
- processing status

Display a table/card view.

Example:

Document	Version	Date	Status	Action
API Documentation	2.0	2024	Outdated	Investigate
API Documentation	3.0	2026	Current	View

8. DOCUMENT PROCESSING

When the user uploads a document:

Upload

↓

Extract text

↓

Clean text

↓

Split into chunks

↓

Generate embeddings

↓

Store vectors

↓

Store metadata

↓

Ready for RAG

Show processing progress.

Handle failures gracefully.

9. RAG IMPLEMENTATION

Implement REAL RAG.

Do not fake RAG.

The system should:

1. Receive the investigation claim.
2. Generate a retrieval query.
3. Search the vector database.
4. Retrieve relevant chunks.
5. Include document metadata.
6. Return evidence to the agent.

Each retrieved item should contain:

- document
- source
- version
- date
- chunk text
- relevance score if available

The retrieved evidence must actually influence the LLM's reasoning.

The LLM should NOT simply answer from its own general knowledge.

10. AI AGENT

Create exactly ONE primary AI Agent:

Knowledge Investigator Agent

This agent must perform an actual multi-step workflow.

Use LangGraph to create a clear state graph.

Agent flow:

START

↓

Understand Claim

↓

Search Knowledge Base

↓

Retrieve Evidence

↓

Inspect Metadata

↓

Evaluate Evidence Sufficiency

↓

If insufficient → Search Again

↓

Compare Relevant Evidence

↓

Send Structured Evidence to LLM

↓

Classify Knowledge

↓

Generate Explanation

↓

Generate Recommendation

↓

END

The agent should have access to tools/functions such as:

search_knowledge()

retrieve_evidence()

compare_evidence()

analyze_claim()

generate_report()

Do not make the agent simply call the LLM once.

The agent must control the workflow.

11. AGENT STATE

Create structured agent state containing information such as:

claim

retrieved_documents

evidence

source_metadata

additional_search_required

comparison

classification

reasoning

confidence

recommendation

human_verification_required

Persist the final investigation result.

12. LLM

Use one LLM.

The LLM must perform meaningful work:

Claim interpretation

Understand what the existing knowledge claims.

Evidence comparison

Compare relevant pieces of evidence.

Semantic contradiction detection

Detect contradictions even if the wording is different.

Classification

Return:

- CURRENT
- OUTDATED
- CONFLICTING
- UNCERTAIN

Explanation

Explain WHY.

Recommendation

Recommend the appropriate next action.

13. LLM SYSTEM PROMPT

Create a strong system prompt along these lines:

"You are the reasoning component of KnowledgeGuard AI.

You must analyze only the evidence supplied by the retrieval and investigation system.

Do not invent evidence.

Do not use your general model knowledge as evidence.

Determine whether the existing knowledge is CURRENT, OUTDATED, CONFLICTING, or UNCERTAIN.

CURRENT means the available evidence supports the existing knowledge.

OUTDATED means reliable newer or applicable evidence indicates that the existing knowledge is no longer current.

CONFLICTING means relevant evidence contains unresolved contradictions.

UNCERTAIN means the available evidence is insufficient or ambiguous.

Always explain the reasoning.

Clearly distinguish retrieved evidence from interpretation.

If the evidence is insufficient, return UNCERTAIN rather than guessing.

Recommend human verification when appropriate."

Implement this properly in the backend.

14. CLASSIFICATION LOGIC

CURRENT

Example:

Existing:

"Python 3.12 is supported."

New document:

"Python 3.12 remains supported."

Result:

CURRENT

OUTDATED

Existing:

"API v2 is recommended."

New:

"API v2 has been deprecated. Use API v3."

Result:

OUTDATED

CONFLICTING

Document A:

"Authentication method A is required."

Document B:

"Authentication method B is required."

Result:

CONFLICTING

UNCERTAIN

The system does not have enough relevant evidence.

Result:

UNCERTAIN

Recommendation:

Human verification required.

15. INVESTIGATION PAGE

Create a dedicated page.

Title:

Knowledge Investigation

Input:

"Enter a claim or question to investigate..."

Example:

Is API v2 still recommended?

When the user starts an investigation, visually show:

✓ Claim understood

✓ Searching knowledge base

✓ Evidence retrieved

✓ Agent investigating

✓ Comparing evidence

✓ LLM reasoning

✓ Generating result

This is extremely important for the college demonstration.

The evaluator should visually see the AI Agent workflow.

16. RESULT PAGE

After investigation, show a professional result card.

Example:

KNOWLEDGE STATUS

OUTDATED

Claim:

API v2 is recommended.

Evidence:

API v2 was recommended in version 2.0 documentation.

New Evidence:

Version 3.0 documentation states that API v2 has been deprecated.

Reasoning:

Newer documentation indicates that the previously stored recommendation is no longer current.

Confidence:

92%

Recommendation:

Review and update the stored knowledge.

Human Verification:

REQUIRED

The exact values must be dynamically generated.

Do NOT hard-code this example.

17. EVIDENCE PANEL

Create a visible evidence section.

For every result display:

Existing Knowledge

The relevant stored information.

Retrieved Evidence

Relevant newer/related information.

Source

Document name.

Version

Document version.

Date

Document date.

Comparison

Explain the difference.

Classification

Current / Outdated / Conflicting / Uncertain.

This evidence-first design is essential.

18. INVESTIGATION HISTORY

Create an Investigation History page.

Store:

- claim
- classification
- explanation
- confidence
- recommendation
- date/time
- evidence references

Allow the user to open previous investigations.

19. SAMPLE DATA

Create realistic sample documents inside:

data/sample_documents/

Create at least four scenarios.

Scenario 1 — OUTDATED

Old API document:

"API v2 is recommended."

New API document:

"API v2 is deprecated. API v3 is recommended."

Expected classification:

OUTDATED

Scenario 2 — CURRENT

Old document:

"Python 3.12 is supported."

New document:

"Python 3.12 continues to receive support."

Expected:

CURRENT

Scenario 3 — CONFLICTING

Document A:

"Authentication requires API keys."

Document B:

"Authentication requires OAuth 2.0."

Expected:

CONFLICTING

Scenario 4 — UNCERTAIN

Create a topic where the available documents do not contain enough evidence to make a reliable conclusion.

Expected:

UNCERTAIN

Do not hard-code expected results into the frontend.

The actual RAG + Agent + LLM pipeline must produce the result.

20. KNOWLEDGE STATUS DEFINITIONS

Display these definitions in the application/help section.

CURRENT

Available evidence supports the stored knowledge.

OUTDATED

Available newer/applicable evidence indicates the stored knowledge may no longer be current.

CONFLICTING

Relevant sources contain unresolved contradictory information.

UNCERTAIN

Available evidence is insufficient or ambiguous.

21. HUMAN-IN-THE-LOOP

Do NOT automatically modify or delete knowledge.

The system should follow:

AI Detection

↓

Evidence

↓

Reasoning

↓

Recommendation

↓

Human Review

↓

Knowledge Update

This is a critical safety feature.

22. DATABASE

Use SQLite.

Create appropriate tables.

At minimum:

documents

id

filename

source

version

topic

document_date

uploaded_at

status

knowledge_chunks

id

document_id

chunk_text

metadata

investigations

id

claim

classification

explanation

confidence

recommendation

human_verification_required

created_at

evidence

id

investigation_id

document_id
chunk_id
evidence_text
relevance_score

23. BACKEND API

Implement clean endpoints:

POST /documents/upload

GET /documents

GET /documents/{id}

DELETE /documents/{id}

POST /investigate

GET /investigations

GET /investigations/{id}

GET /dashboard/stats

Use proper request validation and error handling.

24. SECURITY

Create:

.env

.env.example

.gitignore

Never expose API keys to React.

The frontend must communicate with the backend.

Do not commit secrets.

Use:

LLM_API_KEY=

in .env.example.

25. UI DESIGN

The UI should look like a modern enterprise AI product.

Do NOT make it look like ChatGPT.

Use:

- professional dashboard
- sidebar navigation
- cards
- status badges
- evidence panels
- tables
- charts where useful
- clean typography
- responsive layout
- clear spacing

Suggested sidebar:

KnowledgeGuard AI

Dashboard

Knowledge Base

Investigate

History

About

26. IMPORTANT UX

The application must clearly communicate:

What is happening?

Agent investigation progress.

Why did the system reach this result?

Evidence + reasoning.

Where did the information come from?

Source + date + version.

What should the human do?

Recommendation.

27. ERROR HANDLING

Handle:

- missing API key
- invalid API key
- LLM API failure
- embedding failure
- vector DB failure
- PDF extraction failure
- unsupported file
- empty document
- no relevant evidence
- insufficient evidence
- agent failure
- timeout

Show user-friendly error messages.

Never expose raw backend stack traces in the normal UI.

28. LOADING EXPERIENCE

During document processing:

Uploading...

Extracting text...

Creating chunks...

Generating embeddings...

Indexing knowledge...

Complete

During investigation:

Understanding claim...

Searching knowledge...

Retrieving evidence...

Agent investigating...

Comparing evidence...

LLM reasoning...

Generating report...

Make these states visible.

29. DASHBOARD STATISTICS

Calculate actual values.

Example categories:

Total Investigations

Current

Outdated

Conflicting

Uncertain

Needs Review

Do NOT use fake values such as:

"120 documents"

unless the database actually contains 120 documents.

30. README

Create a complete README containing:

Project

KnowledgeGuard AI

Problem

Explain knowledge decay in modern AI systems.

Solution

Explain the knowledge reliability layer.

Architecture

Explain:

RAG → Agent → LLM → Classification

Technologies

List actual technologies used.

Setup

Exact commands.

Environment Variables

Explain .env.

Running

Exact frontend/backend commands.

RAG

Explain implementation.

Agent

Explain Knowledge Investigator Agent.

LLM

Explain its role.

Demo

Explain the four sample scenarios.

Limitations

Include:

- retrieval limitations
- source quality
- LLM reasoning limitations
- uncertainty
- false positives/false negatives
- human verification requirement

Future Work

Include:

- continuous monitoring
- live data sources
- scheduled investigations
- notifications
- enterprise knowledge bases
- audit logs
- RBAC
- knowledge graphs
- human approval workflows
- specialized agents

31. TESTING

After implementation, actually test the application.

Test:

Test 1

Upload old and new API documents.

Investigate:

"Is API v2 still recommended?"

Verify the result.

Test 2

Investigate the Python support example.

Test 3

Investigate conflicting authentication documents.

Test 4

Investigate insufficient evidence.

Also test:

- file upload
- RAG retrieval
- agent workflow
- LLM response
- database storage
- dashboard statistics
- investigation history
- invalid files
- empty documents
- missing API key

Fix all important errors.

32. VERY IMPORTANT: NO FAKE AI

Do not create fake buttons that only display:

"AI Agent running..."

The underlying agent must actually execute.

Do not hard-code:

"OUTDATED"

for the API example.

The result should come through the implemented workflow.

Do not create a fake RAG component.

The vector database must actually be queried.

Do not create fake confidence values.

Confidence should come from the implemented reasoning approach.

33. DEVELOPMENT PRIORITY

Because this project must be completed quickly, follow this exact priority.

PRIORITY 1

Make these work first:

1. Document upload
2. Text extraction
3. Chunking
4. Embeddings
5. Vector DB
6. RAG retrieval
7. LLM
8. AI Agent
9. Classification
10. Evidence output

Only after these work should you focus heavily on UI.

PRIORITY 2

Then build:

- dashboard

- knowledge base
- investigation history
- polished result page

PRIORITY 3

Only if time remains:

- authentication
- advanced charts
- notifications
- additional document formats
- advanced monitoring

Do NOT sacrifice the AI pipeline for unnecessary UI features.

34. THREE-DAY CONSTRAINT

This is a student project with a strict 3-day development window.

Therefore:

DO NOT add:

- multi-agent architecture
- fine-tuning
- complicated microservices
- Kubernetes
- complicated authentication
- mobile application
- voice interface
- blockchain
- complex knowledge graphs
- unnecessary external integrations

Build a reliable MVP.

35. PROJECT STRUCTURE

Use a clean structure similar to:

knowledgeguard-ai/

1

```
|— frontend/
```

1

└─ src/

1

| $\vdash$ — components/

1

| [└─ pages/](#)

1

```
|  └─ services/
```

1

| `└─ hooks/`

1

```
| └─ App.jsx
```

1

└─ package.json

1

L ...

1

└─ backend/

1

└─ api/

1

└─ agents/

1

 $\vdash \text{rag/}$

1

└─ llm/

1

|— database/

1

└─ models/

1

```
|— services/
```

1

└─ utils/

1

└─ main.py

1

└─ data/

1

└─ sample_documents/

```
|  
├── .env.example  
├── .gitignore  
├── README.md  
└── requirements.txt
```

You may improve this structure if necessary.

36. SYLLABUS CONNECTION

Make the implementation clearly connected to AI syllabus concepts.

The project demonstrates:

Module I

LLMs and intelligent agents.

Module V

Inference and reasoning.

Module VI

Knowledge representation.

Module VII

Planning and agent workflows.

Module VIII

Uncertainty.

Module X

Real-world AI applications.

Mention these connections in the README.

37. FINAL DEMO FLOW

Make sure the finished application supports this demonstration:

Open Dashboard



Show Knowledge Base



Show uploaded old/new documents



Go to Investigation



Enter:

"Is API v2 still recommended?"



Agent starts



RAG retrieves relevant evidence



Agent compares evidence



LLM reasons



Result:

OUTDATED



Show evidence



Show reasoning



Show recommendation



Show Human Verification

This should be smooth enough for a 3–5 minute college presentation.

38. FINAL ACCEPTANCE CHECKLIST

Before declaring the project complete, verify:

- ☐ Application starts successfully.
- ☐ Frontend works.
- ☐ Backend works.
- ☐ Documents can be uploaded.
- ☐ PDF/TXT extraction works.
- ☐ Chunking works.
- ☐ Embeddings work.
- ☐ Vector DB works.
- ☐ RAG retrieval works.
- ☐ LLM works.
- ☐ AI Agent actually executes.
- ☐ Agent performs multiple investigation steps.
- ☐ CURRENT classification works.
- ☐ OUTDATED classification works.
- ☐ CONFLICTING classification works.
- ☐ UNCERTAIN classification works.
- ☐ Evidence is displayed.
- ☐ Source/date/version are displayed.
- ☐ Reasoning is displayed.
- ☐ Recommendation is displayed.
- ☐ Human verification is displayed when appropriate.
- ☐ Investigation history works.
- ☐ Dashboard statistics are dynamic.
- ☐ Error handling works.
- ☐ API keys are protected.

- [] No secrets are committed.
 - [] Sample documents are included.
 - [] README is complete.
 - [] Demo scenario works.
 - [] No core functionality is hard-coded.
-

39. IMPORTANT FINAL INSTRUCTION

Do not stop at planning.

Do not only explain how I could build it.

BUILD IT.

Inspect the current workspace first.

If an existing project exists, determine whether it can be reused or whether a clean structure is better.

Create and modify the necessary files.

Install required dependencies.

Run the application.

Test the application.

Fix errors.

Continue iterating until the core application works.

If one technology causes unnecessary problems, replace it with a simpler reliable alternative while preserving the required architecture.

The final implementation **MUST** contain:

ONE LLM + ONE AI AGENT + RAG

and the actual workflow must be:

**Knowledge → RAG Retrieval → AI Agent Investigation → LLM Reasoning →
Classification → Evidence → Human Verification**

At the end, provide a concise completion report containing:

1. What was built
2. Technology stack

3. Where the LLM is used
4. Where RAG is implemented
5. Where the AI Agent is implemented
6. How to run the project
7. Test results
8. Any remaining issues
9. Exact demo steps
10. Important environment variables required

Do not claim something is working unless you actually tested it.